

Your AI Engineering Department.

AI systems, production software, and infrastructure engineered to deliver results from day one.



Software Development Proposal

Stormo.io Phase 1 Build

AI-Powered Customer Acquisition SaaS Platform

Prepared for:
Stormo.io

Date:
May 25, 2026

 www.credminds.com |  info@credminds.com |  [+92 320 6883136](tel:+923206883136)

This document and the information contained herein are confidential and proprietary to Credminds . It is intended solely for the use of the individual or entity to whom it is addressed. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Before We Begin

Thank you for considering Credminds for your project.

Every engagement we take on is structured around one question: what does success look like for your business? This proposal is built around that answer. Inside, you'll find our understanding of the problem, our approach to solving it, the timeline, and the outcomes we're committing to.

We don't just deliver systems. We deliver results. And we stay accountable until we do.



Ahmad Ashfaq

Founder, Credminds
info@credminds.com

Introduction

About Credminds

We are a team of engineers and product thinkers who handle the full lifecycle of your project. From architecture to deployment, we build AI systems and production software that solve real business problems. We operate as an extension of your team, not an external vendor.



The Numbers Behind Our Work

113+ Number of clients served	85% Client retention rate	13+ Countries Served
--	--	---

Our Clients

			
			

01. Project Understanding

Credminds has reviewed the Stormo.io Developer Proposal Request in full. We understand that this is not a website project. Stormo.io is a production-grade SaaS application with AI at its core, and we are responding to it as such.

What Stormo.io Is

Stormo.io is an AI-powered customer acquisition platform for new ecommerce store owners. The platform guides users from zero customers to consistent weekly growth through a structured system of daily AI-generated actions, automated weekly content, influencer outreach management, and seasonal campaign planning, all without requiring an advertising budget.

The Core Technical Challenge We Are Building Around

The single most critical architectural requirement in this build is the prevention of AI response repetition in the daily action queue. We have read the specification carefully and we agree with the client's position: naive context injection, passing a growing list of raw actions to every Claude API call, is not an acceptable solution. It degrades in cost, quality, and performance as user tenure grows.

Our solution uses a three-layer architecture that keeps token usage bounded and constant regardless of how long a user has been on the platform. We describe this fully in the Technical Screening Questions section of this proposal.

Scope Confirmation

Credminds is proposing to deliver the complete Phase 1 build as specified: all 14 features, all 35 test checklist items, all three authentication methods, both subscription tiers with Stripe, and a fully deployed and signed-off application. Nothing has been excluded from our scope.

02. Recommended Technology Stack

We recommend a Next.js (frontend) + Django REST Framework (backend) stack over the WordPress specification. Our reasoning is specific and architectural, not a matter of preference.

Why Not WordPress for This Build

WordPress is a content management system. Every SaaS requirement in this specification, real-time user state management, streaming AI responses, background job queues for the weekly content pipeline, custom REST API architecture, WebSocket or SSE connections for the conversational onboarding interface, requires working against WordPress's architecture rather than with it. JetEngine and JetFormBuilder add meaningful complexity and abstraction overhead to a build that is fundamentally a custom application. The result would be slower to build, harder to maintain, and more fragile under load.

Our Recommended Stack

The following stack maps directly and cleanly to every feature in the Phase 1 specification. Each component has been selected specifically for this project.

Component	Solution	Rationale
Frontend	Next.js (App Router)	Server components, streaming, App Router architecture
Backend	Django + Django REST Framework	Robust API layer, admin, background tasks
Database	PostgreSQL (Neon)	User records, action history, content, outreach pipeline
Cache / Queue	Redis (Upstash) + BullMQ	Session cache, rate limiting, weekly content job queues
AI Engine	Claude API (openrouter)	All AI features
Payments	Stripe	Intro pricing (\$9 first month), recurring billing, webhooks
Auth	NextAuth.js	Email/password, Google OAuth 2.0, Sign in with Apple

Email	Mailchimp + Transactional API	Milestone emails, weekly digests, deadline reminders
Hosting	Railway (API) + Vercel (Frontend)	Both support long-running API processes, env var mgmt
Vector / Embeddings	pgvector on PostgreSQL	Semantic similarity for AI repetition prevention
Confetti	canvas-confetti (MIT, 3KB)	First Sale milestone celebration screen

03. Technical Screening Questions

All seven screening questions are answered below with full technical specificity.

Q1. Context Injection and AI Repetition Prevention

This is the question we took most seriously in preparing this proposal, and we agree with the client's implicit critique: passing a raw growing history of actions into every Claude API call is not a production-grade solution. It fails on cost, quality, and context limits as user tenure grows. Our approach uses three layers working together.

Layer 1: Semantic Similarity Filtering (Vector Search)

Every generated action is embedded using a text embedding model and stored in pgvector on PostgreSQL. Before generating a new action, the system runs a cosine similarity search against all previous actions for that user. Actions with similarity above a threshold (0.85) are flagged as near-duplicates. Only these flagged candidates, typically 10 to 15, not all 180+, are passed to Claude with the instruction "do not generate anything resembling these." This means Claude sees only the actions most likely to be repeated, keeping the context tight and targeted.

Layer 2: Structured Coverage Map

The user record maintains a compact JSON coverage map that tracks channel usage counts, last-used dates, and outcome signals (positive / neutral / negative / untested). This object is a few hundred tokens at most, regardless of user tenure, and tells Claude everything it needs to know about what has been covered without listing hundreds of individual actions. This object also drives the adaptive sequencing logic described in Q3.

Layer 3: Rolling Compressed Summaries

Every 30 actions, a background job compresses that batch into a short narrative summary: "Days 1–30: Reddit community engagement (6 actions), Pinterest SEO (4 actions), micro-influencer cold outreach (3 actions). Reddit showed positive traffic signals. Influencer outreach had no responses." These summaries stack. By day 180, the user has 6 short summaries, approximately 400 tokens total, rather than 180 raw action records.

The total context injected per API call is bounded at roughly 2,000 tokens regardless of user tenure: store profile (~300), coverage map (~200), compressed summaries (~400), recent 20 actions verbatim (~800), similarity-flagged near-duplicates (~300). This never grows. Repetition prevention is stronger than raw-list injection because vector similarity catches semantic matches that exact-text comparison misses.

Q2. Conversational AI Onboarding

Yes. Credminds has built multi-turn conversational AI interfaces where structured data is extracted and saved to user records in real time across multiple projects, including our Genyios.com platform (described in Q7).

For Stormo's onboarding, we will build a streaming chat interface in Next.js with a Django API layer handling the Claude calls. Each user message is sent to Claude with a system prompt defining the current topic area and instructing Claude to extract specific structured fields from the conversation. As Claude responds, a parallel extraction call identifies newly confirmed data points, store URL, product type, ideal customer description, etc., and writes them immediately to the user record via a PATCH to the profile endpoint. No second step, no form submission.

The frontend maintains a topic progress state that advances when Claude signals topic completion via a structured JSON flag embedded in its response alongside the conversational text. Store URL detection triggers an immediate server-side fetch and Claude analysis, results written to the profile before onboarding ends.

Q3. Adaptive Action Sequencing

The adaptation logic lives in the Django application layer, not in Claude. When a user marks an action complete, they optionally tag an outcome (got traffic, good engagement, no response, etc.). These signals are stored against the action record and reflected in the coverage map described in Q1.

Before generating the next action, the system runs a lightweight aggregation query producing a channel priority vector: which channels have positive signals, which have no signal, which have negative signals, which the user has not tried. This vector is injected into the Claude prompt as structured context: "This user has reported positive results from Reddit community participation (2 instances) and no response from cold email outreach (1 instance). Prioritise Reddit-adjacent and community-based tactics in the next action. Deprioritise cold outreach until further signal."

Claude makes the content decision within these guardrails. The business logic is in Django: auditable, testable, and independent of the AI layer. Claude is generating content within a brief; it is not making architectural decisions.

Q4. Weekly Content Generation Pipeline

We will architect this as a distributed job queue using BullMQ on Redis. The weekly generation process breaks into per-user jobs, each spawning six child tasks: one per content type. Jobs are spread across a rolling 24-hour window at the start of each week to avoid API rate limit spikes. Each job pulls the user's full profile, their previously generated content history (same context injection principle as the action queue), and passes both to Claude with content-type-specific system prompts.

Generated content is written to the database and flagged as available. Users see it on their next login or receive an email notification. Failed jobs retry with exponential backoff. A monitoring dashboard tracks job status per user so generation failures are visible and recoverable without data loss.

Repetition prevention for content follows the same layered approach as the action queue: content history is compressed and Claude is explicitly instructed to avoid repeating structures, angles, or formats from previous weeks. Each generation prompt also includes current date context so Claude naturally references timely angles rather than recycling evergreen frames.

Q5. Platform Recommendation

We recommend Next.js (frontend) + Django REST Framework (backend) over WordPress. Our detailed rationale is in Section 02 of this proposal. In summary: every SaaS requirement in this specification, streaming AI, background queues, real-time state management, custom REST APIs, maps cleanly to this stack without an adaptation layer. WordPress requires fighting against its architecture to deliver each of these requirements.

Q6. Persistent Floating Chat and Dashboard Architecture

The Ask Stormo button is implemented as a fixed-position React component rendered at the root layout level in Next.js App Router, outside any page-specific layout, outside any scrollable container, outside any template that individual dashboard pages control. No page template has any control over elements rendered at root layout. Z-index is set above all dashboard components and position is fixed with bottom-right anchoring. It cannot be displaced.

Conversation context across sessions is maintained by storing the Ask Stormo conversation history in a dedicated `ask_stormo_messages` table on the user record. When the panel opens, the last N messages are loaded and passed as conversation history to Claude. The user's full store profile is injected into the system prompt on every message regardless of session age, ensuring responses are always personalised and specific to their store.

Q7. Previous AI SaaS Work

Credminds has delivered multiple production AI SaaS applications. The two most recent are:

Genyios.com: Multi-Interface AI Agent Platform

Genyios is an AI SaaS platform that allows users to create custom AI agents and connect them to any interface: chat, voice, WhatsApp, email, and Telegram. Key technical elements: Claude API and OpenAI API integration for core reasoning; open-source model support including Qwen for multilingual tasks, VoxCPM for TTS, and Whisper for STT, all hosted on RunPod via vLLM; multi-agent orchestration allowing users to create multiple agents and embed them on external sites; full conversation history management across sessions; and a streaming response architecture for real-time chat and voice. This project required direct handling of context windows, conversation continuity across sessions, and routing logic between models: exactly the patterns Stormo requires.

Shopivibes: AI-Native Loyalty Platform for Restaurants

Shopivibes is a recent AI-native loyalty and engagement platform for the restaurant industry. The platform integrates AI into customer loyalty flows, personalisation, and engagement, requiring reliable production AI API integration at user scale with background job processing

and real-time personalisation. Additional project examples and screenshots are available upon request.

Across these and other projects, Credminds has direct production experience with: Claude API, OpenAI API, open-source models on RunPod/vLLM, OpenRouter for multi-model routing, context window management at scale, conversation history compression, background generation pipelines, and subscription SaaS billing.

04. Delivery Plan

Timeline Overview

Total duration: 8 weeks from signed Statement of Work and receipt of first milestone payment. We have structured the build into four phases, each ending with a working deliverable and a milestone payment.

Start Date On agreement and first payment	Launch Target 8 weeks from start
---	--

Phase Breakdown

Weeks 1–2: Foundation and Core Infrastructure

Project scaffold (Next.js + Django), PostgreSQL schema design, Redis/BullMQ setup, email and social auth (Google + Apple), Stripe integration with introductory pricing logic, user dashboard shell, Railway + Vercel deployment pipeline, CI/CD. Deliverable: authenticated users can register, log in, subscribe, and reach the empty dashboard.

Weeks 3–4 : AI Onboarding and Daily Action Engine

Conversational onboarding interface (5-topic streaming chat, real-time structured data extraction, auto-save to user record), store URL server-side fetching and Claude analysis, daily action queue with full 3-layer context injection architecture, adaptive sequencing logic (coverage map, channel priority vector), Mark Complete, Do This Tomorrow, action storage. Deliverable: full onboarding flow and first action delivered, context injection system live and tested.

Weeks 5–6: Content Pipeline, Outreach CRM, and Dashboard Build

BullMQ weekly content generation pipeline (6 content types, per-user jobs, retry logic), influencer and partnership outreach CRM (7 stages, 3-day follow-up flag, AI follow-up generation), seasonal campaign planner (60-day lookahead), competitor intelligence module, milestone system (9 milestones, confetti on First Sale), full dashboard layout (all 6 nav sections,

all panels), Ask Stormo persistent floating chat. Deliverable: complete beta application, all features functional.

Weeks 7–8: QA, Testing, and Launch Sign-off

Full 35-item checklist execution and documentation, blog setup (4 posts, RankMath SEO configuration, Open Graph), Mailchimp email automation (milestone triggers, weekly digests, deadline reminders), mobile responsive QA, all three auth flows tested, Stripe cancellation and failed payment flows, SSL, cross-browser testing. Deliverable: signed-off test report and live production application.

05. Investment and Payment Structure

Fixed Price Quote

Credminds is quoting a fixed price for the complete Phase 1 build as specified: all features, all 35 test items, fully deployed and signed off. There is no hourly billing and no scope ambiguity. The price below covers everything described in this proposal.

Total Project Investment \$6,520 USD	Billing Model Fixed price: milestone-based
--	--

Payment Milestones

Payments are tied to working deliverables. Each milestone payment is released upon the client's acceptance of the deliverable for that phase.

#	Phase	Deliverables	Payment
M1	Foundation & Core Infrastructure	Next.js app scaffold, PostgreSQL schema, auth (email + Google + Apple), Stripe integration (Starter \$9/\$29, Growth \$49), user dashboard shell, deployment to Railway.	\$1,630 (25%)
M2	AI Onboarding + Daily Action Engine	Conversational onboarding (5-topic chat, real-time save), store URL analysis, daily action queue with full context injection + repetition prevention,	\$1,630 (25%)

		adaptive sequencing logic, Mark Complete / Do This Tomorrow.	
M3	Beta: Content Pipeline + Outreach + Dashboard	Weekly 6-content-type generation pipeline (BullMQ), influencer outreach CRM (7 stages, auto-follow-up flags), seasonal campaign planner, competitor intelligence, milestone system (confetti), complete dashboard layout, Ask Stormo floating chat.	\$1,630 (25%)
M4	QA, Testing & Launch Sign-off	Full 35-item checklist tested and signed off, blog (4 posts + RankMath SEO), email automation (Mailchimp), mobile responsive QA, all edge cases and payment flows verified.	\$1,630 (25%)

No final payment is released until all 35 checklist items are signed off by the client. This is a firm commitment.

06. Next Steps

We are ready to begin immediately upon agreement. The following steps take us from proposal acceptance to development start.

Step 1: Proposal Acceptance

Client reviews and accepts this proposal in writing (email or Fiverr order confirmation). Credminds issues the formal Statement of Work document.

Step 2 : Statement of Work

Credminds produces a detailed SOW document covering full feature specifications, technical architecture decisions, user stories for every feature, acceptance criteria for each milestone, and the complete 35-item test checklist mapped to deliverables. Client reviews and approves the SOW. Estimated turnaround: 2–3 business days.

Step 3: User Story and Scope Approval

Client reviews all user stories and scope definitions in the SOW. Any clarifications or adjustments are resolved before development begins. This step protects both parties from scope ambiguity during the build.

Step 4: First Milestone Payment and Development Start



Stormo.io

On SOW approval and receipt of the first milestone payment (\$1,630, 25%), Credminds begins development. Week 1 commences immediately.

We look forward to building Stormo.io. Please do not hesitate to reach out with any questions before proceeding.

Ahmad Ashfaq

Founder, Credminds

info@credminds.com | www.credminds.com | +92 320 6883136